

OOPS ASSIGNMENT

Question1.

Code :

```
#include <iostream>

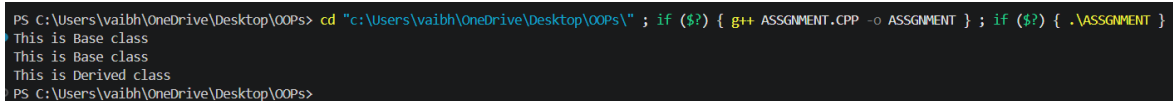
using namespace std;

class Base {
public:
    void showBase() {
        cout << "This is Base class\n";
    }
};

class Derived : public Base { public:
    void showDerived() {
        cout << "This is Derived class\n";
    }
};

int main() { Base b; Derived d; b.showBase();
d.showBase(); d.showDerived(); return 0;
}
```

Output:



```
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> cd "c:\Users\vaibh\OneDrive\Desktop\OOPS\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
This is Base class
This is Base class
This is Derived class
PS C:\Users\vaibh\OneDrive\Desktop\OOPS>
```

Question2.

```
#include <iostream>

using namespace std;

class Base {
    protected:
```

```
int num;

public:

Base() {
num = 100;

}

};

class Derived : public Base { public:

void display() {

cout << "Protected num = " << num << endl;

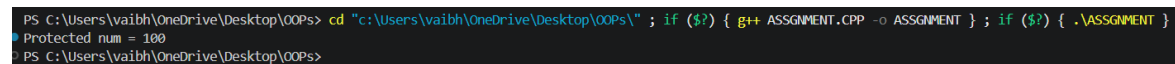
}

};

int main() { Derived d; d.display(); return 0;

}
```

Output:



```
PS C:\Users\vaibh\OneDrive\Desktop\OOPs> cd "c:\Users\vaibh\OneDrive\Desktop\OOPs\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
Protected num = 100
PS C:\Users\vaibh\OneDrive\Desktop\OOPs>
```

Question3.

Code:

```
#include <iostream>

using namespace std;

class Base {

public:

int a = 10;

};

class PublicD : public Base {};

class ProtectedD : protected Base {}; class PrivateD : private Base {};

int main() {

PublicD obj1;
```

```
cout << "Public Inheritance: " << obj1.a << endl;

// ProtectedD obj2; → cannot access a

// PrivateD obj3; → cannot access a return 0;

}
```

Output:

```
PS C:\Users\vaibh\OneDrive\Desktop\OOPs> cd "c:\Users\vaibh\OneDrive\Desktop\OOPs\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
Public Inheritance: 10
PS C:\Users\vaibh\OneDrive\Desktop\OOPs>
```

Question4.

Code:

Single Inheritance:

```
1 #include <iostream>
2 using namespace std;
3
4 class A {
5 public:
6     void showA() {
7         cout << "This is class A\n";
8     }
9 };
10
11 class B : public A {
12 public:
13     void showB() {
14         cout << "This is class B (Derived from A)\n";
15     }
16 };
17
18 int main() {
19     B obj;
20     obj.showA();
21     obj.showB();
22
23     return 0;
24 }
```

This is class A
This is class B (Derived from A)

=== Code Execution Successful ===

Multiple Inheritance:

```
1 #include <iostream>
2 using namespace std;
3 class A {
4 public:
5     void showA() {
6         cout << "Class A\n";
7     }
8 };
9 class B {
10 public:
11     void showB() {
12         cout << "Class B\n";
13     }
14 };
15 class C : public A, public B {
16 public:
17     void showC() {
18         cout << "Class C (Derived from A and B)\n";
19     }
20 };
21 int main() {
22     C obj;
23     obj.showA();
24     obj.showB();
25     obj.showC();
26
27     return 0;
28 }
```

Class A
Class B
Class C (Derived from A and B)

=== Code Execution Successful ===

Hierarchical Inheritance:

<pre>1 #include <iostream> 2 using namespace std; 3 class A { 4 public: 5 void showA() { 6 cout << "This is base class A\n"; 7 } 8 }; 9 class B : public A { 10 public: 11 void showB() { 12 cout << "This is class B\n"; 13 } 14 }; 15 class C : public A { 16 public: 17 void showC() { 18 cout << "This is class C\n"; 19 } 20 }; 21 int main() { 22 B obj1; 23 C obj2; 24 obj1.showA(); 25 obj1.showB(); 26 obj2.showA(); 27 obj2.showC(); 28 return 0; 29 }</pre>	<pre>This is base class A This is class B This is base class A This is class C === Code Execution Successful ===</pre>
--	---

Multilevel Inheritance:

<pre>1 #include <iostream> 2 using namespace std; 3 class A { 4 public: 5 void showA() { 6 cout << "Class A\n"; 7 } 8 }; 9 class B : public A { 10 public: 11 void showB() { 12 cout << "Class B (Derived from A)\n"; 13 } 14 }; 15 class C : public B { 16 public: 17 void showC() { 18 cout << "Class C (Derived from B)\n"; 19 } 20 }; 21 int main() { 22 C obj; 23 obj.showA(); 24 obj.showB(); 25 obj.showC(); 26 return 0; 27 }</pre>	<pre>Class A Class B (Derived from A) Class C (Derived from B) === Code Execution Successful ===</pre>
---	---

Hybrid Inheritance:

<pre>1 #include <iostream> 2 using namespace std; 3 class A { 4 public: 5 void show() { 6 cout << "Class A\n"; 7 } 8 }; 9 class B : virtual public A {}; 10 class C : virtual public A {}; 11 class D : public B, public C { 12 public: 13 void display() { 14 cout << "Class D (Hybrid Inheritance)\n"; 15 } 16 }; 17 int main() { 18 D obj; 19 obj.show(); 20 obj.display(); 21 return 0; 22 }</pre>	<pre>Class A Class D (Hybrid Inheritance) === Code Execution Successful ===</pre>
--	--

Question5.

Code:

```
#include <iostream>

using namespace std;

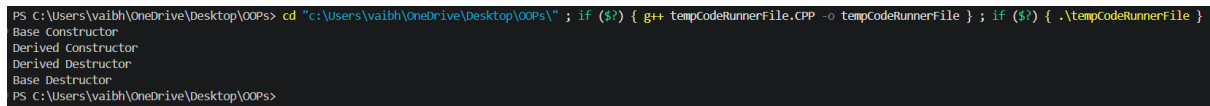
class Base {
public:
    Base() {
        cout << "Base Constructor\n";
    }
    ~Base() {
        cout << "Base Destructor\n";
    }
};

class Derived : public Base { public:
    Derived() {

        cout << "Derived Constructor\n";
    }
    ~Derived() {
        cout << "Derived Destructor\n";
    }
}
```

```
};  
  
int main() { Derived d; return 0;  
  
}
```

Output:



```
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> cd "c:\Users\vaibh\OneDrive\Desktop\OOPS\" ; if ($?) { g++ tempCodeRunnerFile.CPP -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }  
Base Constructor  
Derived Constructor  
Derived Destructor  
Base Destructor  
PS C:\Users\vaibh\OneDrive\Desktop\OOPS>
```

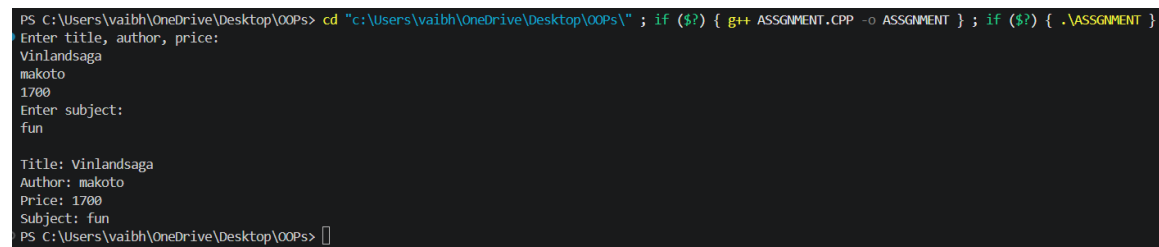
Question6.

Code:

```
#include <iostream>  
  
using namespace std;  
  
class Book {  
protected:  
string title, author; float price;  
public:  
void getBook() {  
cout << "Enter title, author, price:\n"; cin >> title >> author >> price;  
}  
};  
  
class Textbook : public Book { string subject;  
public:  
void getTextbook() {  
cout << "Enter subject:\n"; cin >> subject;  
  
}  
  
void display() {  
cout << "\nTitle: " << title  
<< "\nAuthor: " << author  
<< "\nPrice: " << price  
<< "\nSubject: " << subject << endl;  
}
```

```
};  
  
int main() { Textbook t; t.getBook(); t.getTextbook(); t.display(); return 0;  
  
}
```

Output:



```
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> cd "c:\Users\vaibh\OneDrive\Desktop\OOPS\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }  
Enter title, author, price:  
Vinlandsaga  
makoto  
1700  
Enter subject:  
fun  
  
Title: Vinlandsaga  
Author: makoto  
Price: 1700  
Subject: fun  
PS C:\Users\vaibh\OneDrive\Desktop\OOPS>
```

Question7.

Code:

```
#include <iostream>  
  
using namespace std;  
  
class Speedometer { protected:  
  
int speed;  
  
public:  
  
void setSpeed(int s) { speed = s; }  
  
};  
  
class FuelGauge { protected:  
  
int fuel; public:  
  
void setFuel(int f) { fuel = f; }  
  
};  
  
class Thermometer { protected:  
  
int temperature; public:  
  
void setTemp(int t) { temperature = t; }  
  
};  
  
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer { public:  
  
void display() {  
  
cout << "Speed: " << speed << " km/h\n"; cout << "Fuel: " << fuel << " %\n";  
  
cout << "Temperature: " << temperature << " C\n";  
  
}
```

```
}  
};  
  
int main() { CarDashboard car; car.setSpeed(80); car.setFuel(60); car.setTemp(35); car.display(); return  
0;  
}
```

Output:

```
PS C:\Users\vaibh\OneDrive\Desktop\OOPs> cd "c:\Users\vaibh\OneDrive\Desktop\OOPs\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }  
Speed: 80 km/h  
Fuel: 60 %  
Temperature: 35 C  
PS C:\Users\vaibh\OneDrive\Desktop\OOPs> █
```

Question8.

Code:

```
#include <iostream>  
  
using namespace std;  
  
class LibraryUser {  
protected:  
string name; int id;  
public:  
void getUser() {  
cout << "Enter name and ID:\n"; cin >> name >> id;  
}  
};  
  
class Student : public LibraryUser { int grade;  
public:  
void getStudent() {  
cout << "Enter grade:\n"; cin >> grade;  
  
}  
  
void display() {  
cout << "Student: " << name << " ID: " << id << " Grade: " << grade << endl;  
}  
};  
  
class Teacher : public LibraryUser { string dept;
```



```
public:

void getTeacher() {

cout << "Enter department:\n"; cin >> dept;

}

void display() {

cout << "Teacher: " << name << " ID: " << id << " Dept: " << dept << endl;

}

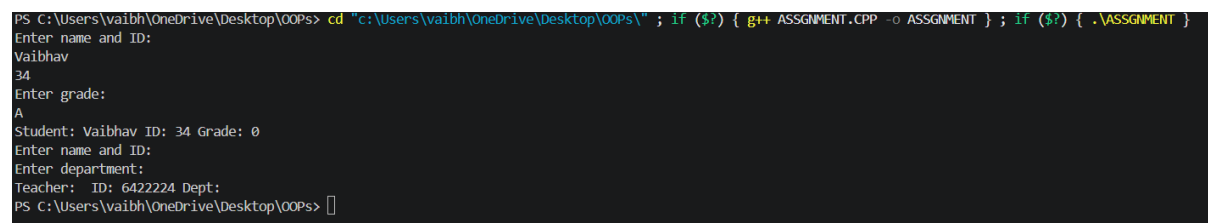
};

int main() { Student s; Teacher t; s.getUser(); s.getStudent(); s.display();

t.getUser(); t.getTeacher(); t.display(); return 0;

}
```

Output:



```
PS C:\Users\vaibh\OneDrive\Desktop\OOPs> cd "c:\Users\vaibh\OneDrive\Desktop\OOPs\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
Enter name and ID:
Vaibhav
34
Enter grade:
A
Student: Vaibhav ID: 34 Grade: 0
Enter name and ID:
Enter department:
Teacher: ID: 6422224 Dept:
PS C:\Users\vaibh\OneDrive\Desktop\OOPs>
```

Question9.

Code:

```
#include <iostream>

using namespace std;

class Vehicle {

protected:

string make, model; int year;

public:

void getVehicle() {

cout << "Enter make, model, year:\n"; cin >> make >> model >> year;

}

};

class Truck : public Vehicle { protected:

int load_capacity; public:
```

```
void getTruck() {
    cout << "Enter load capacity:\n"; cin >> load_capacity;
}

};

class RefrigeratedTruck : public Truck { int temperature_control;
public:
void getRefrigerated() {
    cout << "Enter temperature control:\n"; cin >> temperature_control;
}

void display() {
    cout << make << " " << model << " " << year
    << " Load: " << load_capacity
    << " TempCtrl: " << temperature_control << endl;
}

};

int main() { RefrigeratedTruck r; r.getVehicle(); r.getTruck(); r.getRefrigerated();
r.display(); return 0;
}
```

Output:

```
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> cd "c:\Users\vaibh\OneDrive\Desktop\OOPS\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
Enter make, model, year:
toyota inova 2010
Enter load capacity:
344
Enter temperature control:
45
toyota inova 2010 Load: 344 TempCtrl: 45
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> []
```

Question10.

Code:

```
#include <iostream>

using namespace std;

class Person {
protected:
string name; public:
void getPerson() {
```

```
cout << "Enter name:\n";

cin >> name;
}

};

class Staff : virtual public Person { protected:
int emp_id; public:
void getStaff() {
cout << "Enter employee ID:\n"; cin >> emp_id;
}
};

class Student : virtual public Person { protected:
int student_id; public:
void getStudent() {
cout << "Enter student ID:\n"; cin >> student_id;
}
};

class TeachingAssistant : public Staff, public Student { public:
void display() {
cout << "Name: " << name
<< " EmpID: " << emp_id
<< " StudentID: " << student_id << endl;
}
};

int main() { TeachingAssistant ta; ta.getPerson(); ta.getStaff(); ta.getStudent(); ta.display();
return 0;
}
```

Output:

```
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> cd "c:\Users\vaibh\OneDrive\Desktop\OOPS\" ; if ($?) { g++ ASSIGNMENT.CPP -o ASSIGNMENT } ; if ($?) { .\ASSIGNMENT }
Enter name:
Vaibhav
Enter employee ID:
23
Enter student ID:
12
Name: Vaibhav EmpID: 23 StudentID: 12
PS C:\Users\vaibh\OneDrive\Desktop\OOPS> 
```